

Session-based Music queue Reordering System Using Skip Behaviour as Implicit Feedback

Data Science MSc

William McKenna

School of Computing and Mathematical Sciences,
Birkbeck, University of London

Word Count: 11,979

1. Introduction

1.1 The Problem

Consider the following scenario; an individual opens Spotify to listen to music. Unsure of what they want to listen to, they enter a playlist composed of their favourite tracks and click play. Regardless of whether the listener is in shuffle mode the songs follow each other with no decided-upon order. Still unsure of what they want the user skips a track, then a couple of tracks that suit their mood come on, before being interrupted by a track that does not, forcing another skip. This is a common occurrence, as reinforced by this research with the sample population experiencing an average of a 33.1% skip rate across their filtered listening history. This research intends to utilise these skip signals to model the users current listening mood, by calculating the probability of the remaining songs being skipped and reorder the queue accordingly to improve the listening experience.

1.2 Aims & Objectives

The aim of this research is to produce a model that will rank the unplayed tracks in a listening queue, based on the outputted skip-probabilities. These skip-probabilities will be inferred using audio and behavioural features present in the played tracks (context). Four different models will be evaluated over three loss functions to deduce what combination fits best for this scenario. Further analysis will then be conducted to highlight the impact of contextual length on the results.

1.3 How this research will be conducted

This research aims to model this behaviour by treating skips as implicit feedback, regarding skips as ‘positive’ feedback and full plays as ‘negative’ feedback. This action can be treated as implicit as skips are actions that are already an integral part of the listening experience. Therefore, they will not add any additional cost to the listener. A limitation must be addressed, as skip behaviour cannot be exclusively attributed to features we can derive. There are human factors that can’t be modelled from the data available. This will be expanded upon in Chapter 7. For the purposes of this research, skips are defined as a conscious choice by the listener to move on to the next song, the exact definition will be stated in Section 3.5.

The skip-behaviour will be captured by four different models that will output skip-probabilities that will be used to re-rank the unplayed portion of the queue. Logistic Regression and Extra Trees classifier were chosen as baselines. Two sequential models were added to investigate whether treating the session as a sequence improves the ranking: A Long Short-Term Memory network (LSTM) and a Self-Attentive Sequential Recommendation network (SASRec), Kang and McAuley (2018), which were compared against each other to evaluate a recurrent model vs. a self-attentive one.

Three different loss functions were applied to the sequential models to investigate different ways of penalising errors made by the model. The first is Binary Cross-Entropy (BCE) which is used as a baseline to compare against the two novel loss functions proposed below. The second is an adapted version of the Distributionally robust Rényi Loss proposed by Zhang et al. (2025). This was designed for a collaborative filtering problem so has been adapted from the reference code to suit this research. The final, Positionally-Weighted Time-Sensitive loss (PWTS), is a custom loss proposed in this research tailored for this application. It consists of three weights to measure the following: position of the track in the session, time of the skip during the song and the user’s historical skip rate of that track.

1.4 Overview of the results

All four models were evaluated over the same dataset, with the two sequential architectures scored on each of the loss functions described previously. This gives eight trained arms against a randomly ordered baseline. Tuning for the sequential networks was carried out over 1080 configurations, with each chosen model being retrained over 5 random seed arms before evaluation. The test set was held out until the final stage to avoid data leakage.

Every model improved upon the randomly ordered baseline of 0.6881 NDCG@5 (the selection metric). The best arm, LSTM with loss of PWTS, scored 0.7846 a gain of 0.0965 (31% of the available headroom left by the baseline). The Rank First Skip metric, included to intrinsically measure what the model hopes to achieve, states that with random ordering the user listens to an average of 3.7 tracks before skipping. The best performing model ensures the user listens to an average 5.15 songs, showing an improvement of 1.5 songs. Whilst this may seem like a modest improvement, it's a 41% increase. Moreover, this is a measurement over a single reordering, in deployment the queue is reordered after each skip. Given a median session length of 31 tracks, with random ordering the listener will experience eight interruptions throughout the session, whereas with the model applied the listener will only be interrupted six. The LSTM model performed better than the SASRec model across every matched loss and likewise for the PWTS against the other 2 losses (according to NDCG@5).

A separate analysis into the effect of context length on the performance of the model confirmed the hypothesis: Increased context length increases the predictive performance of said model over the AUC metric, however, this performance saturates within the first 40% of tracks.

1.5 Comparable Works

There are similar pieces of work that tackle a comparable problem, however there are limitations to each of these as outlined below.

Moor et al. (2023), demonstrates the commercial significance of a similar task; a large scale study at Spotify showing that reordering a playlist queue according to sequential skip preferences measurably reduces skip rates across users. The skip-probability is calculated from historical behaviour patterns, with all tracks being treated as equivalent data points. This model can therefore identify that listeners skip at particular points in a session but cannot identify where a user wants to skip because of the characteristics a particular track holds.

Meggetto et al. (2023) researched which features are most valuable when predicting the skip rate of a track. Dividing inputs into behavioural, content and contextual groups and finding that behavioural features dominate. This research, uses `reason_start` as one of their inputs and is their strongest predictor. This introduces a limitation, as this feature is unknowable until the preceding track has finished so deems the model unusable as a live predictive system for this task. Additionally, the task carried out is framed as a binary classification, predicting whether the next track is likely to be skipped, this works as research into the effect of the input features, but has little use as a deployable model.

Jin and Lee (2024) produce the most similar work, combining audio and contextual features evaluated on the Music Streaming Sessions Dataset, Brost et al. (2019). However they set the context window at ten tracks for every session, which is an inaccurate portrayal of how the model would operate in deployment. This research also frames the problem as a classification task with no ranking problem introduced and the loss function is confined to standard binary cross-entropy.

Seshadri et al. (2024) frame skip prediction as ranking and utilise the SASRec architecture. The limitations lie in their uniform treatment of each position in the session, and their allowance of each query track to attend another query track.

This research frames skip prediction as a ranking problem over the unplayed portion of a queue. It is evaluated using ranking metrics, it restricts every model input so that it may only use information knowable at each position in the session, it proposes a loss function tailored to this exact problem and ensures all decisions are made such that the model is usable as a live predictive system.

1.6 Report Structure

Chapter 2, Literature Review

Contains a critical analysis of all comparable, related and source papers, linking it to the methods and analysis carried out later in the report.

Chapter 3, Proposed Methods

Outlines the data-processing pipeline from raw datasets to clean, anonymised, filtered sets ready for input into the model. Provides high-level explanations and justifications for the models/loss functions used and their architectures.

Chapter 4, Implementation

Outlines how the models were put into practice, what steps were taken to optimise the performance of said models and any hurdles that were overcome in doing so.

Chapter 5, Evaluation & Testing

This chapter will give an overview of the decisions taken in the evaluation and testing stages and the motivations behind them.

Chapter 6, Results & Discussion

Provides an in-depth analysis of all findings derived from the evaluation stage.

Chapter 7, Conclusion & Deployment

Summary of the report, identification of any potential future revisions.

Bibliography + Appendix

2. Literature Review

This chapter examines the works that prompted the design of this research, with each section addressing corresponding design decisions described in Chapter 3.

2.1 The General Task of Music Recommendation

Music recommender systems have typically been built utilising contrasting collaborative filtering combined with content-based filtering, as per Schedl et al. (2018). The collaborative filtering infers listener preference by learning behavioural similarities between users, while content-based filtering, recommends using the acoustic properties of the tracks.

Collaborative filtering relies on an accumulated interaction history and therefore Schedl et al. (2018) is assuming that user preference is in a stable state, allowing signals recorded over months or years to predict what a listener wants to hear at a particular moment. This author suggests that a listener's acceptance of a track varies with their mood, recent activity & what has just been played, long-term data collection struggles to capture this. The unit of this model is therefore chosen as the session rather than the user. Therefore, the model is conditioned on what is observed in the current session rather than a listening profile built over the user's history.

2.2 Skip Behaviour as an Implicit Signal

Explicit feedback is defined as a deliberate action taken to state a preference. This requires effort from the user which results in the signals being typically sparse. Implicit feedback is an action that is inferred from a behaviour that occurs regardless. Skips belong to the latter, as they are generated through ordinary use of a streaming service. These signals are valuable for the application of music recommendation as they are timestamped, common and impose no additional effort to the user.

The use of implicit signals as noisy indicators of preference was formalised by Hu et al. (2008). They address the problem that an absent interaction is indistinguishable from an item that was never seen, weighting observations by confidence so that the strength of the available data is encoded. However, with this application, every track is observed whether skipped or not so the non-skipped tracks form a complementary class, rather than having to be inferred. Therefore, the weighting is unnecessary, and the skip can be the event that the model predicts, contradicting the usual convention where consumption is treated as the positive signal.

This research is designed according to the fact that skip signals should not be treated equivalent to each other. Yi et al. (2014) propose that dwell time (the amount of time a user 'dwells' on a particular item) is significant and provides more to the model than a simple binary interaction. Wu et al. (2020) extend this argument by treating unusually short dwell times as actively negative rather than weakly positive. In the context of music streaming, this implies that a track abandoned after four seconds should be treated differently to a track that is skipped after two minutes. The other skip-prediction works outlined in Section 1.5 fail to capture this information and thus inspires the inclusion of the time-weight in the PWTS loss function proposed in section 3.2.3.

2.3 Data Collection & Processing

Other works carried out on the topic of skip prediction have largely used the Music Streaming Sessions Dataset (MSSD), Brost et al. (2019), published in conjunction with the WSDM 2019 challenge AICrowd (2019). This challenge proposed the task format on which subsequent research has converged upon. That is that sessions are divided into an observed portion (played tracks) and a held-out portion (unplayed tracks), with the model predicting skipping behaviour in the second given the behaviour in the first. This dataset was withdrawn from public access in July 2024, resulting in

this research using the more limited dataset collected independently from volunteers as outlined in section 3.1.1.

The construction of the sessions from a raw listening history requires the sessions be split upon rules dictating when they start and finish. The widely used convention of thirty minutes being used as the standard length of time to indicate online inactivity, originates from work carried out by Catledge and Pitkow (1995).

Meggetto et al. (2023), while investigating the importance of each feature on the MSSD identified a temporal data leakage in two features: the length of the session and the reason the track ends. Both features supply the model with data that no live system could possess. This informs both the causal construction of the historical skip rate defined in Section 3.1.6 and the detection of the leak in the pipeline in Section 4.4.

The concern of temporal data leakage also applies when splitting the dataset into the training, validation and testing sets. Meng et al. (2020) demonstrates that datasets that have been randomly split, allow a model to be trained on interactions that occur after those it is evaluated on. This supports the decision to split the dataset chronologically and per-user.

2.4 The Usable Reasons Behind a Skip

In this research, the decision of what is used to inform the model about whether a track will be skipped, is answered in two parts. Firstly, which observable properties of the track, session or listener provide useful information and secondly, which of these observations are available to every track in the unplayed portion.

Meggetto et al. (2023) attempt to provide an answer to the first part. They did so by estimating feature importance using SHAP and then validating by ablation, removing one feature category at a time. They then carried out paired t-tests on the resulting change in accuracy. The removal of the reason a track started cost 28% of the mean accuracy, the largest cost by far. Additionally, they found that removing the content features had no significant change on the accuracy.

This work contradicts prior work with van den Oord et al. (2013) demonstrating the positive effect of content-based features and Oramas et al. (2017) showing that audio features contribute when combined with other categories while being insignificant alone. The work of Jin and Lee (2024) also support this, demonstrating that a combination of audio features and contextual features produce a more accurate prediction than contextual features alone. This research doesn't attempt to resolve this, choosing to include the available audio features combined with behavioural and contextual features.

The second part of this decision is not as widely disputed. Upon analysing the input features provided by the MSSD, Meggetto et al. (2023) removed session length & reason end due to temporal data leakage. The reason a track began was retained as this is applicable to the task outlined in their research, predicting the skip outcome of the next track. This research argues that this does not provide any use to the listener, beyond selecting a random song to take its place if it is predicted as a skip. To aid improvement of the listening experience this research ranks all remaining tracks in the queue to place the track with the lowest skip probability as the next track. The reason a track begins is unknown to all unplayed tracks as it is dependent on the track preceding it to finish. Therefore, it was removed as a feature for this model.

This decision is reinforced by the concept of train/serve skew outlined by Zinkevich (2017), in which a model relies on a signal present in the observed training data and is absent at inference, thus scoring well in evaluation while becoming unusable in production. Xu et al. (2020) addresses this problem by including the value of these 'privileged' features to train a teacher then removing it to produce a deployable student, making it available at training time to aid explanation of interactions between

tracks but absent when it comes to evaluation. This route was considered for this research, but the decision was made to focus on the comparison of architectures and loss functions.

2.5 Sequential Architectures for Session-Based Recommendation

The inclusion of the sequential architectures in this research is what allows the system to model inter-positional relationships within a session rather than prediction being made with each position treated equally. The use of recurrent networks for session-based recommendation models was established by Hidasi et al. (2016). Adapa (2019) extended this research using bidirectional LSTMs in the WSDM Cup 2019 challenge. Bidirectionality is not suited for this task as it allows a track's score to depend on tracks appearing later in the queue, this is why the decision was made to include a unidirectional LSTM model in Section 3.2.1.

The work of Kang and McAuley (2018) introduced the use of a Self-Attentive Sequential Recommendation (SASRec) model to replace recurrency, allowing each position to attend all preceding positions weighted by learned relevance. Then, a causal mask was implemented preventing any position attending its own future. Sun et al. (2019) removed that constraint with their proposed model BERT4Rec, which attends positions bidirectionally under a masked-item objective carrying a learned token for positions with a withheld value. As before, the bidirectional attention is unsuitable for this task, but the masking device inspired the use of the three-state status channel defined in Section 3.2.1, where unplayed tracks carry an unknown state rather than being omitted.

2.6 Loss Functions for Noisy Implicit Feedback

All the comparable literature mentioned so far penalises prediction errors using the unweighted BCE loss. This treats every training row as carrying equal influence.

The work carried out by Zhang et al. (2025) addresses the construction of a loss which captures the noisy signals typical with implicit labels. They do this by identifying weaknesses in the loss functions used in collaborative filtering: Softmax loss is overly sensitive to false negatives and Cosine contrastive loss discards most negative samples. They propose that replacing the underlying divergence measure with the Rényi divergence controls how the negatives are weighted, producing the Distributionally robust Rényi Loss function (DrRL) used in this research. This is relevant to skip prediction as errors occur in both directions: a completed play can reflect distraction rather than preference and a skip may be a product of circumstances unrelated to the track. The loss function derived by Zhang et al. (2025) was used as a reference for this research, however it is designed for collaborative filtering so has been adapted as described in Section 3.2.3.

Jeunen and Goethals (2019) trained a single recurrent network with a custom weighted loss for the WSDM Cup 2019 challenge. They established that the reweighting of the cross-entropy loss is a viable solution for this task. The basis for this is provided by Hu et al. (2008) who weighted the squared error (minimised with alternating least squares) with a confidence term that reflects the strength of the evidence behind it. Furthermore, Järvelin and Kekäläinen (2002) propose that the positions earlier in a sequence should matter more than the positions later in the sequence (applying a logarithmic discount over rank), combined with the dwell-time findings of Yi et al. (2014) and the historical skip rate defined in 3.1.6 we can capture the strength of each of the signals. These pieces of literature provide the groundwork for the custom loss, PWTS proposed in Section 3.2.3.

2.7 Evaluation & Testing

Ranking tasks are typically evaluated using the area under the ROC curve, computed from the Mann-Whitney rank statistic Hanley and McNeil (1982) and is scored over all outputs at once. Zhu et al. (2017) establish the convention of calculating a user weighted AUC which quantifies the quality of

intra-user order by averaging the user weighted scores over all users. This issue applies directly to queue reordering, with ordering decisions being made in a single session and is applied in Section 5.2.

AUC by session, however, has limitations in this application as not all positions should be treated equally as put forward by Järvelin and Kekäläinen (2002). Evaluating evenly over the whole set of remaining songs in a queue does not produce a score that reflects the improvement of listening experience. Therefore NDCG@5 is used as the primary selection metric as it weights the earlier positions higher in the scoring process.

The decision to evaluate each sequential model over an average of five randomly selected seeds is reinforced by the work of Reimers and Gurevych (2017), who found that randomselection is enough to present the same recurrent network as high-performing or mediocre.

3. Proposed Methods

3.1 Data Collection & Processing

This section will provide an in-depth description of the motivations and implementations of the data-processing pipeline and process carried out to collect the data. The pipeline is run by a single file '*data_processing_main.py*' and consists of five stages to produce three datasets that are ready to be trained, validated and evaluated by the models described in Section 3.2.1.

3.1.1 Data Collection Ethics

The participant group for this research consists of volunteers that the author reached out to via email/social media & private messages. The number of volunteers culminated at 33, which met the minimum set-out in the research proposal of 30 individuals. Each volunteer was presented with an information sheet outlining how their data would be stored, anonymised, used and disposed of as well as a consent form which permits the author to use their data within the confines detailed in the information sheet. Volunteers were instructed to download their Spotify Extended Streaming History Export (ESHE) from their Spotify account and then email it to the researcher.

Volunteers were offered an analysis of their listening history as a gesture of gratitude for providing their data (as outlined in the ethics application (appendix C)). Once sent, the following identifying data points were removed: name & email address. Each dataset was saved as a collection of JSON files in an indexed folder and each user's IP address is deleted upon execution of the data processing pipeline.

The audio features dataset was downloaded from a public dataset on [huggingface.co ozefe](https://huggingface.co/ozefe), (2025) which consists of audio features for 256 million tracks.

Upon the execution of the *data_processing_main.py* script a random uuid is assigned to each folder and is then broadcasted to each row in the combined JSON files. This ensures that there is no traceable attachment of the volunteer to their dataset. The raw exports persist on the researchers' local machine until completion of the research as described in the ethics application (appendix C).

3.1.2 Data Processing Pipeline

The data pipeline is executed by a single file ‘*data_processing_main.py*’. This consists of five scripts; These are outlined in further detail below:

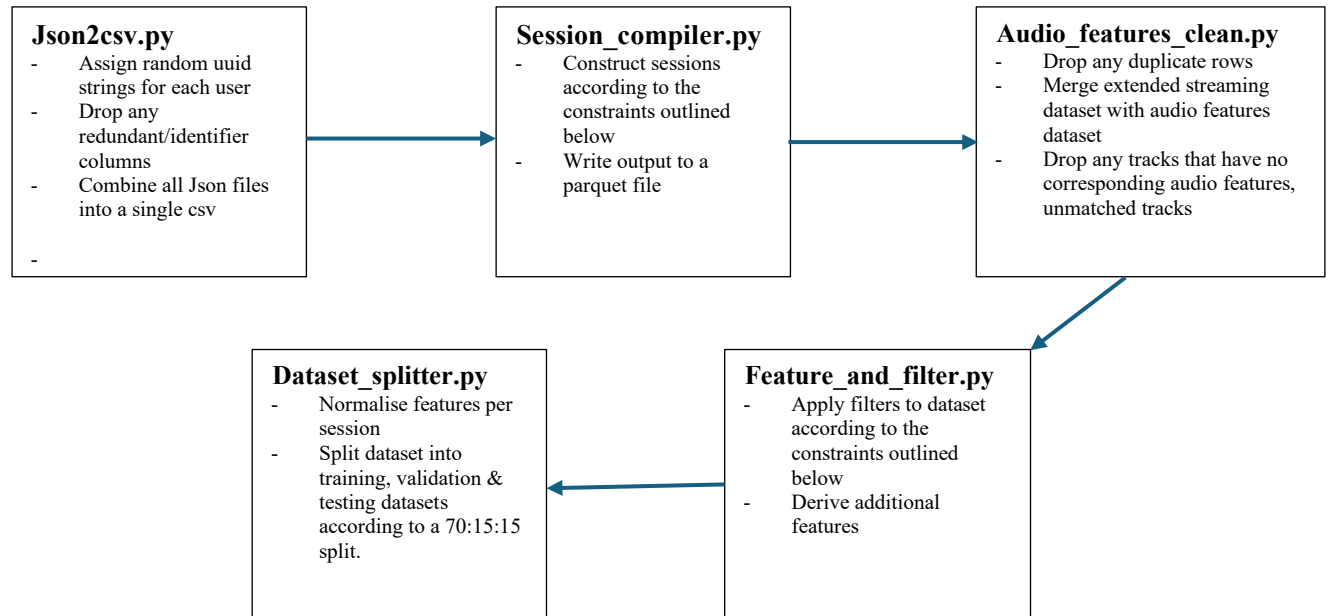


Figure 3.1.) Diagram to outline the order of the pipeline with brief descriptors of the purpose of each sub-script

The ordering of the above pipeline was decided upon for the following reasons:

- **Sessions must be built before the merge:** If unmatched tracks are dropped before the construction of sessions artificial gaps will be created, potentially creating a false boundary between sessions.
- **Filtering after merge:** 528k tracks are unmatched after the join. If we filter before the join, we could have sessions containing less tracks than the required amount.
- **Normalisation after merge and filtering:** Features must be normalised on the dataset that the model will be operating upon.

3.1.3 Constructing Sessions

Research carried out by Catledge, L. D. and Pitkow, J.E. (1995) resulted in the length of 30 minutes being defined as the standard length of time between events such that the period be deemed as inactive. This standard has gone on to be used in most online applications. I therefore deduced that this would be a sensible time constraint for the period of inactivity needed to define a new session of listening.

However, it must be addressed that this introduces some inaccuracy as it is only an approximation and there will be some sessions that have been inappropriately split in two.

3.1.4 Merging of Datasets and the definitions of conventional audio features

The audio features dataset and the combined streaming histories dataset is merged to assign each of the tracks in the ESHEs with its corresponding audio features. The audio features that were assigned are as follows:

Feature	Definition	Range	Observed	Mean (sd)
tempo	Estimated overall tempo in beats per minute	> 0 BPM	0.000 - 249.438	119.262 (28.791)
mode	Modality of the track: 1 for major, 0 for minor	{0, 1}	0 - 1	0.596 (0.491)
danceability	How suitable a track is for dancing. Derived from tempo, rhythm stability, beat strength and regularity	0.0 - 1.0	0.000 - 0.988	0.600 (0.175)
energy	Perceptual measure of intensity and activity	0.0 - 1.0	0.000 - 1.000	0.605 (0.221)
loudness	Overall loudness averaged across the track, in decibels	-60 - 0 dB	-60.000 - +4.638	-8.343 (4.614)
speechiness	Presence of spoken words; above 0.66 indicates a track made almost entirely of speech	0.0 - 1.0	0.000 - 0.966	0.097 (0.102)
acousticness	Confidence measure that the track is acoustic	0.0 - 1.0	0.000 - 0.996	0.290 (0.304)
instrumentalness	Confidence that the track contains no vocals; above 0.5 is intended to indicate an instrumental track	0.0 - 1.0	0.000 - 1.000	0.115 (0.265)
liveness	Probability that the recording features an audience; above 0.8 strongly suggests a live performance	0.0 - 1.0	0.000 - 1.000	0.187 (0.159)
valence	Musical positiveness conveyed by the track, from negative to positive	0.0 - 1.0	0.000 - 1.000	0.478 (0.247)

Definitions are sourced from Spotify's documentation and presented in the dataset card of the audio features dataset, ozefe (2025).

Although the audio features dataset is extensive, there are some tracks present in the ESHEs that are not included. To ensure there are no invalid entries going into the model, the two datasets were merged utilising an inner join. As a result, 528,415 entries were dropped (shown in Table 3.2), this is likely due to them being recent releases/audiobooks/niche tracks. The reason for this is Spotify limiting access to these audio features as November 2024, this was the largest dataset available at the time of writing.

3.1.5 Skip Definition

Table 3.1) Breakdown of reasons for a track ending

Label	Reason the track ended	Plays	Share of label
Skipped	fwdbtn	1,314,673	99.43%
Skipped	clickrow	7,528	0.57%
Not skipped	trackdone	1,420,950	63.98%
Not skipped	endplay	611,914	27.55%
Not skipped	backbtn	82,107	3.70%
Not skipped	unexpected-exit-while-paused	37,437	1.69%
Not skipped	logout	34,180	1.54%
Not skipped	remote	21,524	0.97%
Not skipped	unknown	5,436	0.24%
Not skipped	unexpected-exit	3,694	0.17%
Not skipped	trackerror	1,441	0.06%
Not skipped	popup	1,409	0.06%
Not skipped	other (6 values)	902	0.04%

The definition of a skipped track is a track that is deliberately ended before completion. Two values in reason_end satisfy this definition, fwdbtn & clickrow. As we can see in Table 3.1, 'fwdbtn' accounts for 99.4% of all skipped tracks, where as 'clickrow' accounts for 0.6%.

Two reasons were considered for inclusion in the skipped label, ultimately, they remained as not-skipped:

- 'endplay': includes closing the app, pressing pause, loss of connection etc. As none of these are the result of the user not liking a track it remains as not-skipped.
- 'backbtn': Although there are cases when a user does not like a track and rewinds to the previous track, this is rare. This is more likely a product of users wanting to restart the track or re-listen to the previous track. Given more time & resources it would be worth an analysis into whether the use of this button indicates preferences of the track itself or surrounding tracks.

3.1.6 Additional Feature Derivation

In addition to the audio features provided by the ozefe dataset, we can derive some behavioural features to aid the capture of the listeners' in-session habits. The calculated features are as follows, 'historical_skip_rate', 'historical_artist_skip_rate', 'is_repeat_track', 'same_artist_as_prev', also the binary feature 'shuffle' is inherited from the ESHEs.

- **Historical skip rate:** This feature is calculated by ordering all listens for a user by its timestamps. Then using an expanding mean, for each track, the historical skip rate is calculated on all prior listens (after filtering), this is to avoid future leakage.
- **Historical artist skip rate:** This feature was introduced to resolve a cold-start problem created by the historical skip rate. After initial analysis, 32.2% of listens had a historical skip rate of 0. However, 62.2% of these 0 skip rate listens had historical listens of the same artist. This provides information to 710,376 rows that otherwise would have had no historical data. Aside from this, it also provides extra context intuitively, as if an individual tends to skip a certain artist regardless of the track, they are likely to skip it again.
- **Is repeat track:** A binary feature; True if this track has already been played in this session | False otherwise.
- **Same artist as prev:** A binary feature; True if the previous track was by the same artist | False otherwise.
- **Shuffle:** A binary feature; True if the user started the track in shuffle mode | False otherwise.

‘actively_selected’ & ‘reason_start’ were initially included in the code but ultimately did not end up being an active feature in the models, more detail will be provided on this in 3.2.2.

3.1.7 Data Filtration

The subsequent step in the pipeline is the filtering of the dataset. These entries are obtained using the constraints defined below:

- **Session Length ≥ 7 tracks:** Sessions with fewer tracks cannot support a context/query split with sufficient data points for training and evaluation.
- **Unique Artists > 1 :** A listening session containing no more than one artist indicates that the user has already actively selected what they want to listen to and defeats the purpose of this model.
- **Number of Skips ≥ 1 :** A listening session with 0 skips provides no information for the model to train on, it cannot reorder a queue of songs based on negative feedback alone.

Table 3.2) The effect of each operation on the quantity of data that is usable for the model

Stage	Rows	Sessions	Rows kept	Sessions kept
Raw exports	5,242,623	—	100.0%	—
Session construction	5,242,623	297,141	100.0%	100.0%
Audio feature join	4,714,208	281,365	89.9%	94.7%
Validity filter	3,546,781	108,417	67.7%	36.5%

As shown above, in Table 3.2, the validity filter excludes 24.8% (1.2m) of remaining rows and 61.5% (173k) of remaining sessions. Although this is a significant amount it is necessary to ensure the model is being trained on reliable data.

3.1.8 Normalisation

Of the 10 audio features utilised in the model, 9 are continuous values with a variety of means and standard deviations, therefore we must normalise them so that they can be compared against each other in the neural network models. To carry out the normalisation a z-score is computed for each row within a session. The decision to normalise within each session is intuitive given the problem at hand, predicting in-session listening habits. Consider the following example: A song with energy 0.5 will seem energetic in a session where the average energy is 0.3 but low energy in a session where the mean is higher.

3.1.9 Splitting of the Dataset

The final stage of the pipeline is splitting the dataset into training, validation & testing datasets. This is done using a per-user chronological 70:15:15 split.

The split was done per user because of the large variance in contribution across the 33 volunteers, with the number of sessions per user varying from 94 up to 11,261. This ensures that the validation and testing datasets have as varied a sample population as possible. If it wasn’t split by user, it would be possible for the testing stage to test on one or two users, skewing the results and producing inaccurate conclusions.

The dataset was split chronologically to attempt to mimic real world constraints by reducing future leakage. Otherwise, the model would be attempting to train on later listens and test on earlier ones, predicting the past from the future.

Table 3.3) The split of datasets after processing

Split	Users	Sessions	Rows	Skip rate
Training	33	75,875	2,537,905	38.15%
Validation	33	16,262	510,661	34.62%
Testing	33	16,280	498,215	35.59%
All	33	108,417	3,546,781	37.28%

3.2 The machine learning models

This section will describe the design decisions made throughout the construction of this model and the motivations behind these decisions.

3.2.1 The Models

For this research four models were selected for evaluation and comparison; Logistic Regression, Extra-Trees Classifier, Long Short-Term Memory model (LSTM), and an adapted Self-Attentive Sequential Recommendation Model (SASRec).

The inclusion of the baseline models (Log regression & Extra-Trees) is backed by Ferrari Dacrema et al. (2019) who state the neural recommender models often fail to beat simpler models when tuned equally. Each model is included to isolate a particular capability; Log regression captures linear relationships, Extra-Trees Classifier adds non-linear relationships, LSTM adds recurrence and SASRec adds self-attention. All four models are evaluated on the same dataset using the same functions, imported from the '*Model_lib.py*' script. The latter three models were tuned over various hyper-parameters. The tuning grid for each model was decided upon using exploratory analysis on a small dataset.

Logistic Regression

Hyper-Parameter	Value	Reason
solver	lbfgs	Default
max_iter	1000	To avoid ConvergenceWarning upon execution
C	1.0	Large number of training rows, so would be inconsequential

Extra-Trees Classifier

Hyper-Parameter	Tuned values	Selected Values on best arm
max_depth	8, 10, 12, 15	8
min_samples_leaf	10, 20, 30	20
n_estimators	100, 200, 300	100

Ultimately the Extra-Trees classifier model was chosen over the Random Forest model suggested in the proposal, to improve run time and reduce variance. This is done because the Extra-Trees classifier selects split thresholds at random rather than optimising.

LSTM

Hyper-Parameter	Tuned values	Selected Values on best arm
hidden_units	64, 128, 256	128
dropout_rate	0.1, 0.3, 0.5	0.3
num_layers	1, 2, 3	1
lr	0.001, 0.0001	0.001

The LSTM receives the same 15 input features, the same skip-status channel, the same loss functions and the same evaluation method as the self-attentive model. The two models only differ with the encoder, therefore any difference in results can be attributed to this.

Recurrent models have been utilised to capture listening habits before, with Adapa (2019) using an encoder-decoder recurrent network in the WSDM Cup sequential skip prediction challenge, achieving a mean accuracy of 0.604 MAA. Hidasi et al. (2016) carried out research into recurrent models used in the more general session-based recommendation area. However, both focused on sequence classification rather than addressing the reordering of the remaining queue.

In selection of which recurrent model to use, the vanilla RNN was excluded due to the length of the sessions in the dataset. This model, therefore, requires a gated architecture to ensure the gradients persist. Bi-directional RNNs were excluded as a bi-directional network, when reading the query segment would allow a track's score to be defined by tracks later in the queue.

The front end is identical to that of the self-attentive model. Each track's feature vector is projected into the hidden space by a single linear layer, summed with its status embedding, and passed through dropout. Dropout is applied explicitly to the projected input rather than relying on the dropout parameter of nn.LSTM, which is silently ignored when num_layers is set to one. Without this, the tuned dropout rate would have had no effect across the single-layer third of the hyperparameter grid. Sequences are packed with enforce_sorted=False and unpacked with total_length pinned to the collated batch width, so that per-chunk predictions concatenate correctly during evaluation, where chunks have differing maximum lengths.

SASRec

Hyper-Parameter	Tuned values	Selected Value
hidden_units	64, 128, 256	64
dropout_rate	0.1, 0.3, 0.5	0.1
num_blocks	1, 2, 3	1
num_heads	4, 8, 16	4
lr	0.001, 0.0001	0.001

Fixed Parameters:

- maxlen = 200
- norm_first = True

The second neural network replaces recurrence with self-attention, meaning each position in a session can attend to an earlier position rather than reaching it through a hidden state. The architecture present in this research was adapted from Kang and McAuley (2018), their model was designed to predict the next item based on an item catalogue. This scenario requires an alteration on this, as the model must assign a skip probability to each track in the query segment. The implementation was adapted from

Huang (2020), a pytorch version of the original TensorFlow release, used under the Apache 2.0 license.

In the reference model each item is represented by a learned embedding, resulting in the architecture relying on item identity. This does not function in this scenario as the model must score tracks that may not appear in the training data. The embedding stage is thus replaced by a linear projection of each track's 15 dimension feature vector in the hidden space. The scaling of the embeddings in the original model can therefore be removed, with the decoding of the embeddings in the output layer being replaced by a linear layer that outputs a single logit score for each position and therefore track.

Due to each track carrying an observed skip outcome, negative sampling is unnecessary so is replaced by binary supervision at every query position. Two components were added (skip-status channel & context/query attention mask), which will both be explained in more detail below. All remaining components are inherited from the reference; point-wise feed-forward blocks, layer normalisation, structure & the multi-head attention layers.

Skip-Status Channel

Both neural models receive a second input channel encoding what is already observed for each session. Each position is assigned one of three states; skipped, not-skipped, unknown. This is embedded in the hidden space and summed with the projected feature vector. Tracks before the context boundary carry their observed outcome whereas tracks in the query segment are stated as unknown. The embedding for the unknown positions is initialised to zero. An unplayed track is therefore in the model as a feature vector alone. The skip-status channel also functions in a way to capture skip momentum in the context portion of the session. Meggetto et al. (2021) state that a listener in a restless disposition skips several tracks in succession, independent of the track's features. No input feature encodes this but if each position is assigned its observed outcome the model is free to infer what aspects of the observed outcomes it deems useful.

Context/Query Attention Mask

In the reference model a causal mask is applied, this ensures that a position can only attend to itself and the positions preceding it. However, the reference model outputs a single score at a time. This model is scoring the whole query segment simultaneously so therefore would introduce circularity, for example if the track at position 7 is attending to position 5 (both in the query segment), this arrangement of the query segment is about to be altered by the model which will change the scoring, which will in turn affect the arrangement. The mask is therefore altered so that each position in the query segment may only attend to itself and the context, as demonstrated below.

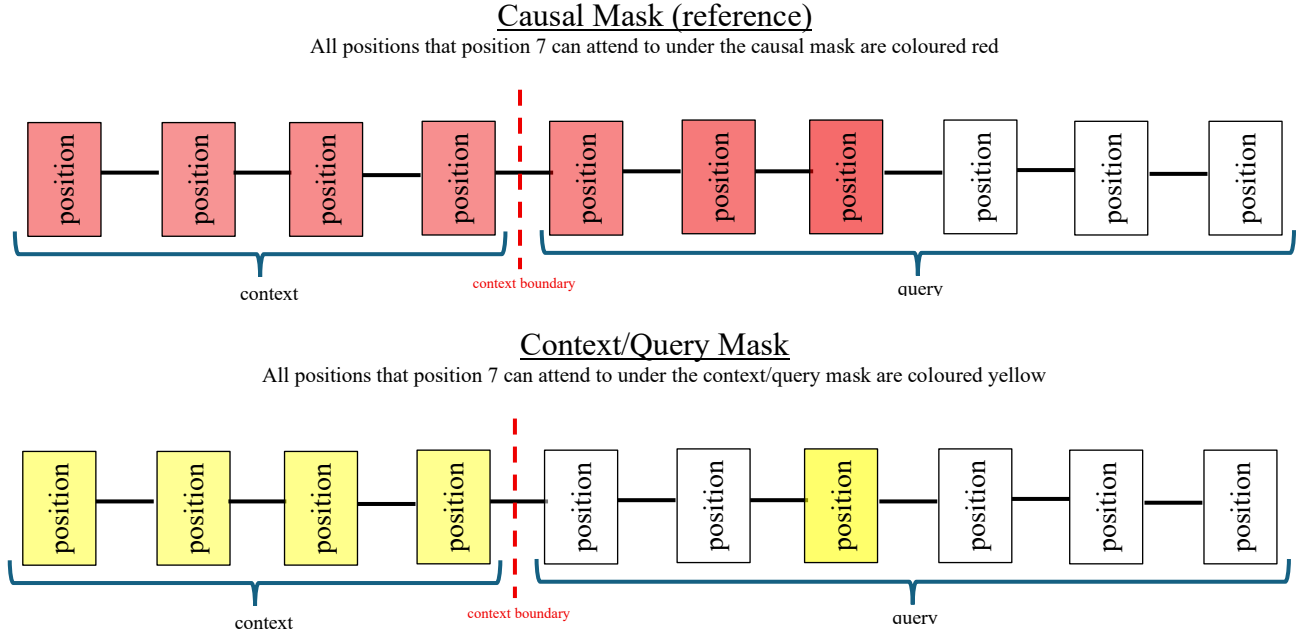


Figure 4.1) Demonstration of the masks used in the reference model vs. this paper's model

3.2.2 The Unknown Feature Problem

Initially, two input features were included in addition to the 15 described in Section 3.1.6; ‘actively_selected’ & ‘reason_start’. The core reasoning behind the removal, is that it’s impossible to know these features unless the track has ended, therefore ‘reason_start’ will be undefined in production. In an offline setting, when simulating the context/query split, this information is available to us as they are all tracks that have been played in reality. In real life applications the query segment of the session will have no data available for ‘actively_selected’ & ‘reason_start’, as ‘reason_start’ is only defined once the track before has been fully played, ‘actively_selected’ is derived from ‘reason_start’ so is also excluded on the same logic.

If the model was trained on this ‘offline-available’ data, it would perform well in the evaluation stage and learn to rely on it yet it will not be usable for production, resulting in an unusable model This is an example of train/serve skew as defined by Zinkevich (2017).

3.2.3 Loss Functions

This research also compares three different loss functions over the two neural models. Binary Cross-Entropy (BCE) as a baseline, a position-weighted time-sensitive loss function (PWTS), a custom loss function derived specifically for this application and an adaptation of the Distributionally Robust Rényi Loss (DrRL) of Zhang et al. (2025), adapted from the author’s reference implementation.

BCE Formula:

$$L_{BCE} = - \frac{\sum (y_i \cdot \log(\sigma(z_i)) + (1 - y_i) \cdot \log(1 - \sigma(z_i)))}{N}$$

Where:

N is the number of tracks in the query position

z_i is the outputted logit score from the model

y_i is the observed outcome ($y_i \in \{0,1\}$)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Initially, both the sequential models output probabilities calculated by the sigmoid function, however due to the requirements of the DrRL, both models now output unbounded logit scores. DrRL is defined over unbounded ranking scores so therefore if confined to a unit interval its gradient vanishes as the probabilities converge to one.

DrRL is a ranking loss function rather than a per-track one (as with BCE). With S^+ and S^- representing the skipped and non-skipped tracks in the query segments, and z for the logit score the model outputs for each track, we have:

$$u_j = c \cdot \max(z_j - \beta, 0) + \varepsilon \text{ for each } j \in S^-$$

$$R = \left(\frac{\sum_{i \in S^-} u_j^{\gamma^*}}{|S^-|} \right)^{\frac{1}{\gamma^*}}$$

$$L_{DrRL} = -\frac{\sum_{i \in S^+} z_j}{|S^+|} + R$$

Where:

- γ is the Rényi order and $\gamma^* = \frac{\gamma}{(\gamma - 1)}$ is its conjugate exponent
- η is defined as the divergence budget and is set to $\eta = 0.1$, bounding how far the worst-case distribution can depart from the observed one. With $c = \frac{(\gamma - 1)}{\eta^{\frac{1}{\gamma - 1}}}$ scaling the penalty to it.
- β is the learnable threshold above which a non-skipped track begins to contribute set at 0.33 (the average skip rate over the population)
- $\varepsilon = 0.1$ keeps u_j positive, so a gradient survives when no track exceeds β

The first term in the L_{DrRL} equation raises the scores of the skipped tracks and the second, R , penalises only those non-skipped tracks scoring above β . The second is a power mean of order γ^* , which is what the Rényi order controls. As the Rényi order approaches 1, γ^* grows and R converges on the maximum loss across non-skipped tracks. γ was tuned alongside each model's hyperparameters, selecting 2.0 for the recurrent model and 4.0 for the transformer. B is learned by a separate gradient taken before each batch's loss is computed.

Three changes were needed to adapt the reference implementation to this task. Every unplayed track in the query segment carries an observed outcome, so sampling negatives from a catalogue is replaced by a Boolean mask over the non-skipped tracks, with the penalty computed across all. Scores are read directly from the output layer, as there is no item embedding present in either model. Skipped tracks form the positive class, as the model is trained to predict skips.

PWTS multiplies the baseline loss by three distinct weights;

$$PWTS = BCE \cdot w_{position} \cdot w_{time} \cdot w_{historical}$$

With each of the weights defined as below

- Position Weight:

$$w_{position} = 1 + e^{-3p}$$

Where p is the % of the progression through the session.

This decays exponentially as the session progresses. This is derived from the idea that an error in prediction earlier on in the query segment matters more to the listener than a later error.

- Time Weight:

$$\text{When } t < 0.1: w_{time} = 2 - \frac{t}{0.1}$$

$$\text{Otherwise: } w_{time} = 1$$

Where t is the percentage of the track listened to before the playback ends.

This weight decays from 2 linearly until 10% of the track has been listened to before skipping, after that the weight defaults to 1. If a listener skips a track in the early stages of a song that should indicate a stronger skip than a later skip. ' t ' is derived from the features ' ms_played ' & ' $track_length$ ', as ms_played is an unknown feature at the time of prediction it cannot be included in the model inputs so including it in the loss function is a way to include it in the model.

- Historical Weight:

$$w_{historical} = 1 + \sigma(6(h - 0.5))$$

Where h is the historical skip rate, σ is the logistic function, 0.5 is the midpoint of the curve & 6 is the steepness of the curve.

The historical weight increases the penalty on a track that the user has habitually skipped as a False Negative (playing a song that the user historically dislikes) is an active annoyance to the listener and will disrupt the listening experience whereas a False Positive (not playing a song the user historically fully plays) is passive, more of a missed opportunity.

All constants used in the three PWTS weights were decided upon using exploratory analysis using the best model from the evaluation stage. The three weights are normalised to a unit mean to preserve the overall loss magnitude of the BCE.

4. Optimisation & Review of Implementation

This chapter will outline the steps taken to maximise the performance of the pipeline and address any and all hurdles encountered in the construction of said pipeline.

4.1 Structure

All components that are shared are written in a single file '*Model_lib.py*', included is the dataset and collation logic, both sequential architectures, the training loop and all evaluation metrics. The evaluation metrics ensure that all four models are evaluated using the exact same code.

4.2 Performance Optimisation

AUC Score

The AUC score is computed once per session, per split, per epoch. This culminates in tens of thousands of calls during a single run of the sequential models. Therefore, rather than using `sklearn.roc_auc_score`, `fast_auc` is utilised using the Mann-Whitney ranking statistic as per Hanley and McNeil (1982). This avoids per call validation and removes the need to construct the ROC curve. This agrees with the scikit-learn implementation and runs significantly faster.

LengthBucketSampler

The length of sessions varies hugely, ranging from 7 tracks to 1148. When a batch is assembled, it is then padded to the length of the longest session, this results in most of the tensor being padding, adding unnecessary computational power and time invested to the tuning of these models. LengthBucketSampler groups sessions of similar length to the same batch to reduce the amount of padding present, this is based on work done by Sutskever et al. (2014).

Audio Feature Dataset

The audio feature dataset is 12.05GB across ten parquet files and is stored locally alongside the streaming histories. Rather than combining the files and reading over the whole dataset for each streaming history to find the matching tracks, each parquet is read with the eleven required columns and filtered against the stored list of tracks present in the volunteer data. This is appended to a list containing a subset of the parquets processed so far and subsequently the parquet is released before opening the next one. This ensures the maximum memory taken up in the merging of the datasets is the current parquet and the stored subset as opposed to the full 12GB.

The forward pass in the evaluation stage is chunked in a similar way to optimise peak memory.

4.3 Experiment Execution

The tuning stage consists of 1,080 configurations and is distributed as SLURM job arrays on the cluster provided by Birkbeck University. Each arm of the tuning has a different number of configurations ranging from 54 for the LSTM with BCE and 486 for the SASRec with DrRL.

Each task evaluates a single configuration, writes it to a csv file, named accordingly and then concatenated with the other tasks of that arm upon completion. This avoids any risk of interleaving between tasks or partial writes and improves the simplicity of reviewing whether all tasks were completed rather than a silent failure. An example of where this aided troubleshooting is below:

- Accidental deletion of a virtual environment terminated 712 tasks with exit code 127, as each file was written individually and named descriptively it was a simple task to rerun the tuning on the missing tasks

4.4 Errors Discovered and Remedied

Data Processing

In construction of the historical skip rates, to chronologise the sessions, the data frame was initially sorted on `'session_id'`. For users with 10 or more sessions this results in session number 10 being placed before session number 2, so the historical skip rate was being calculated from sessions from the future and therefore silently inflating the results. This was corrected by sorting the dataframe on `'user_id'` & `'ts'`.

DrRL Bounded Outputs

DrRL is defined over unbounded outputs from the model. Initially the DrRL was applied to sigmoid outputs resulting in a loss of the gradient and a decline in the performance of the loss function as the model improved. The output sigmoid was removed from both sequential models so that they would produce unbounded logits.

Inconsistent Reporting

evaluate.py initially saved the last seed's per session scores rather than correctly saving the five-seed mean as reported in the terminal output. This was spotted due to a cross-reference from the terminal to the saved csv and has since been corrected.

Learnable DrRL Threshold Stepped Twice

The threshold β in the DrRL loss is a learnable parameter with its own SGD optimiser. Because the model's Adam optimiser is constructed from the model's parameters combined with the loss function's, β was also registered with Adam. The gradient computed for the SGD step was therefore consumed a second time. Adam normalises its update with the gradient's running magnitude, so moves a parameter according to its own 1×10^{-3} lr rather than the lr of 1×10^{-6} intended for β . Therefore, the threshold drifted a lot faster than designed, regularly changing which non-skipped tracks contributed to the penalty and destabilising the loss. This was corrected by clearing the gradient after the SGD and β is now detached from the batch loss so that it is only moved by the SGD.

5. Evaluation & Testing

This chapter will give an overview of the decisions taken in the evaluation and testing stages and the motivations behind them.

5.1 The Metrics

There are 6 different metrics present in the final results table in chapter 7. Below is the description for each and the reasoning as to why they were chosen:

NDCG@5 (selection)

This is the metric chosen to be the selection metric (the metric that decides the optimal parameters during tuning). This is computed over the reordered query segment, which has been ordered ascending by skip-probability with a track being treated as relevant if it was not skipped. This metric was chosen as the selection metric over AUC, as the AUC score evaluates the ordering of the entire query section equally. The NDCG@5 is more suitable for this application as it weights the early positions higher. This is important, as in real-life deployment the queue order would change after each skip. When evaluating the performance of the model after five tracks have played, it is not relevant whether the 42nd track is ranked correctly. This feature of the real-life deployment also justifies the selection of capping the NDCG at 5; the average skip rate across all filtered sessions is 37.3%.

Therefore, under random ordering the average user skips every $\frac{1}{0.37} = 2.7$ songs. As the session progresses and the model reorders the queue this number should increase, explaining the pick of 5. This metric is sourced from the work of Järvelin and Kekäläinen (2002).

AUC (session)

AUC is a pairwise statistic that calculates the probability of a skipped track receiving a higher skip probability than the skip probability of a non-skipped track. This is the secondary evaluation metric, demoted from the selection metric for the reasons described above.

NDCG@10

This metric was included as a robustness check to evaluate whether the performance of the model holds up when evaluated over a larger number of predictions.

P@5

Included as an easily interpretable metric that any reader who is not well versed on the subject can understand. i.e. if the model scores $P@5 = 0.8$, 4 out of the next 5 songs in the queue would be played to completion by the listener.

RankFirstSkip

This metric measures how far the listener gets in the queue before they hit a track that is observed as skipped. This is directly the metric that the queue reordering is trying to optimise.

Random Floor

This is included so that each metric can be compared against itself when the queue is randomly reordered.

5.2 Evaluating per Session

All metrics in the evaluation and testing stages are computed per session with the final scores being an average over all sessions in that dataset. Originally the models were evaluated over the whole datasets, however this introduces a variety of problems specific to this application.

The selection metric NDCG becomes redundant when used over the whole dataset; NDCG is a ranking metric that orders the predictions and compares them to the observed outcomes. When applied to the whole dataset, the metric will be comparing the five songs with the lowest skip rate in all sessions across the dataset. This measures a scenario that never occurs in real world deployment so will not paint an accurate depiction of the performance of the models.

Evaluating AUC over the whole dataset introduces a similar problem. AUC is a pairwise evaluation metric, calculating the probability that a randomly chosen positive outranks a randomly chosen negative. When applied to the whole dataset there is a substantial number of pairs to select. This however, results in positive tracks being compared to negative tracks which are in a different session. Again, this does not accurately depict the application of this model in real-life deployment. This decision significantly reduces the AUC score, however, it is a necessary decision to ensure we are evaluating a real-life scenario.

5.3 Multi-Point Splitting

Each session is evaluated on up to five context/query split points. The scores are then mean averaged within the session and then averaged across all sessions, to ensure that each session contributes equally.

Multiple points were chosen, as if we took a single random split point per session, the score of each session would depend on where the split landed in the session. For example, take 2 random split points in a single session. Before the first, let's say we only have one skip present in the context section, and in between the first and second split we have four skips. If we score on the second split, we are evaluating the model with five times the amount of information compared to the first split. This is reinforced by the context analysis carried out in Section 6.3, which as a byproduct of the analysis, states that the average standard deviation of NDCG@5 scores between the five splits is equal to 0.1364 and the average range of scores across a session is 0.3297. The spread between the best performing sequential model and worst is 0.0267, which means the variance of the model's performance would be swallowed by the variance between random split choice. Split-points are selected at five evenly spaced points dependent on the length of the session. This ensures that there is no random seed generation and that each run is reproducible.

5.4 Research Design

Evaluation was carried out at three distinct stages; First the model was tuned over 1,080 configurations, training on the training set and testing on the validation set, to distinguish the best combination of hyper-parameters for each model-loss combination. The model then retrained on the training set and evaluated again on the validation dataset with five randomly selected seeds using the optimal configurations defined in the previous step. The resulting models were then scored against the test split. Therefore, keeping the test set unseen until final evaluation and ensuring no future leakage.

Selection at each stage used NDCG@5 over a moving window consisting of 3 epochs. This is to counteract any noise creating an unnatural peak and therefore falsely claiming over-performance of a model. In stage two early stopping is triggered when the smoothed value has not improved for 5 five epochs or the number of epochs hits 100. For stage one, early stopping is triggered at five epochs and 20 max_epochs for lr=0.001 and 40 max epochs for lr=0.0001, this is to reduce tuning time and computational costs during the tuning stage.

6. Results & Discussion

This chapter will present the results of the tuning, training and testing stages described in Chapter 5, and discuss what conclusions we can draw from them. All results are compared against the random floor mentioned in Section 5.1, rather than against zero. The random floor is calculated for each metric over a randomly ordered query set. This resulted in the following values:

AUC	NDCG@5	NDCG@10	P@5	RankFirstSkip
0.5028	0.6881	0.7438	0.6254	3.6986

6.1 Hyper-Parameter Tuning

Table 6.1) The configuration selected for each neural arm

Model	Loss	Hidden units	Layers / blocks	Heads	Dropout	Learning rate	γ	Validation NDCG@5
LSTM	BCE	256	1	-	0.1	0.0001	-	0.7990
LSTM	PWTS	128	1	-	0.3	0.001	-	0.7937
LSTM	DrRL	256	3	-	0.1	0.001	2	0.7982
SASRec	BCE	256	1	16	0.1	0.001	-	0.7834
SASRec	PWTS	64	1	4	0.1	0.001	-	0.7824
SASRec	DrRL	128	1	4	0.1	0.001	4	0.7762

6.2 Final Comparison

Table 6.2) Final Results Table with average scores and stddevs taken over 5 seeds for the neural arms.

Model	Loss	Seeds	Sessions	AUC	NDCG@5	NDCG@10	P@5	RankFirstSkip
LSTM	PWTS	5	11,695	0.6426 $\pm$ 0.0029	0.7846 $\pm$ 0.0039	0.8214 $\pm$ 0.0033	0.695 $\pm$ 0.0032	5.1498 $\pm$ 0.0317
LSTM	BCE	5	11,695	0.6427 $\pm$ 0.0075	0.78 $\pm$ 0.005	0.8175 $\pm$ 0.0038	0.6913 $\pm$ 0.0036	5.1386 $\pm$ 0.0975
LSTM	DrRL	5	11,695	0.6448 $\pm$ 0.021	0.7782 $\pm$ 0.01	0.8165 $\pm$ 0.0071	0.6874 $\pm$ 0.0049	5.1995 $\pm$ 0.3219
SASRec	PWTS	5	11,695	0.635 $\pm$ 0.0045	0.7791 $\pm$ 0.0033	0.8159 $\pm$ 0.0027	0.6907 $\pm$ 0.0024	4.9133 $\pm$ 0.0529
SASRec	BCE	5	11,695	0.6333 $\pm$ 0.0039	0.7772 $\pm$ 0.0034	0.8144 $\pm$ 0.0028	0.6897 $\pm$ 0.0026	4.872 $\pm$ 0.0411
SASRec	DrRL	5	11,695	0.6137 $\pm$ 0.0153	0.7598 $\pm$ 0.0111	0.8011 $\pm$ 0.0083	0.6774 $\pm$ 0.0084	4.6912 $\pm$ 0.1833
Extra-Trees	—	1	11,694	0.5987	0.7585	0.8008	0.6789	4.623
Logistic regression	—	1	11,694	0.5948	0.7522	0.7963	0.6751	4.5505
Random ordering	—	1	11,694	0.5028	0.6881	0.7438	0.6254	3.6986

All eight models were scored on the held-out test set when selection was complete.

6.2.1 Overview of Results & Baselines

Every model exceeded the random floor by a significant margin. The best arm, LSTM with PWTS Loss scored 0.7846 NDCG@5. Read as an absolute value, this gives the impression of a near perfect model, however if we compare to the random floor, we see that the gain, although measurable, is not

substantial. In fact, this arm gains 0.0965 over random which is 30.9% of the 0.3119 of headroom the random floor leaves available.

Most of this gain is not due to sequential modelling. Logistic Regression closes 66% of the distance between the floor and the best model and Extra Trees a further 7%. Leading to the conclusion that 73% of the improvement above randomness can be attributed to non-sequential modelling. This is precisely the argument that Ferrari Dacrema et al. (2019) propose in their work and reinforces the inclusion of the baseline models.

6.2.2 Comparing Neural Architectures

The LSTM beats the SASRec at matched loss on every metric. On NDCG@5 the margins are 0.0028 (BCE), 0.0055 (PWTS) & 0.0184 (DrRL). The decision is unanimous across all fifteen paired comparisons.

However, there are limitations within the design of these architectures that skew these results. As in theory the SASRec should be better suited to tackle this problem.

The context/query mask addressed in Section 3.2.1 prevents SASRec from being conditioned on the realised order of the unplayed queue, as discussed prior, this is integral to the logic of this application. However, the LSTM's hidden state is not required to condition on this, so therefore is modelling an easier problem.

Recurrent models have the assumption built into their architecture that sequences are processed in order and that recent positions weigh more than earlier. Self-Attentive models do not assume this. If we apply the findings of Meggetto et al. (2021) in regard to the importance of skip momentum, this partly explains the stronger performance of the recurrent model. Had the dataset been larger the flexible nature of SASRec's architecture may have learned to capture this along with other inter-positional relationships that the LSTM is not equipped to capture. This hypothesis is reinforced by the work of Dosovitskiy et al. (2021) who state that transformer models underperform models with strong prior assumptions on moderate datasets yet overtake them at a substantially larger scale. However, this was not achievable within the scope of this research so has been penned for future extensions of this work.

6.2.3 Comparing Loss Functions

PWTS is the best loss function over both the neural architectures under the selection metric of NDCG@5. It beats BCE by 0.0046 on the LSTM and by 0.0019 on the SASRec and beats the DrRL by 0.0064 on the LSTM and 0.0193 on the SASRec.

The secondary metrics reinforce this when applied to the SASRec, with PWTS beating BCE by 0.0017 AUC and 0.041 Rank First Skip. However, this does not hold with the LSTM, with the difference between the baseline and the PWTS being marginal. This supports the claim that PWTS improves ranking when applied to the SASRec consistently across metrics, with the LSTM supporting the improvement just when using the selection metric.

DrRL is decisively the weakest loss on SASRec, scoring 0.0193 NDCG@5 and 0.0213 AUC below PWTS. The LSTM, however, is more mixed. LSTM with loss of DrRL scores the lowest compared to the other losses on LSTM when measuring with the selection metric. However, when measuring using the secondary metrics AUC & Rank First Skip it performs much better than the PWTS. This can be explained by the instability the DrRL produces, with a standard deviation of 0.32 for Rank First Skip and 0.021 for AUC (10 and 7 times the stddev of PWTS on LSTM respectively).

6.2.4 Rank First Skip

The Rank First Skip metric is the metric that this entire project aims to increase, the number of songs before the listener first skips a song. As stated by the random floor, under random ordering the listener reaches its first skipped track after 3.7 tracks. Under the best model the Rank First Skip is increased to 5.15, indicating a measurable increase in listening experience due to this model.

6.3 Context Analysis

This analysis was carried out to investigate the effect of context length on the performance of the model. Simply, to evaluate how the effectiveness of the reordering of the remaining tracks increased as the listener's listening session progressed.

6.3.1 Method

This analysis scores each session at several context/query split points. Intuitively, the longer the context, the more skip signals that will be inputted into the model, therefore more information to train the model on. This introduces the main hypothesis of this analysis that the farther the listener progresses through the session, the better the model will perform.

The analysis was run on the test set (after model evaluation) using the best scoring model over the selection metric: LSTM with PWTS loss, averaged over its five seed checkpoints. Each session was scored at five evenly spaced split points, with each a split only being admitted if the query window had at least 5 tracks in it and covered at least 10% of the session. This is so that later split points were not evaluated on small query windows. This filter yielded 8,807 sessions with a median session length of 31 tracks.

NDCG@5 was not chosen for this analysis as the model is already improving upon the random floor in the first five tracks as well as it can. The errors that remain are tracks that are deemed unskippable according to all observed input features but are skipped due to features not available from this dataset. This is reinforced by Table 6.3, demonstrated by a constant lift value from the random floor with NDCG@5. Therefore, AUC was chosen as it will quantify the improvement the model experiences due to increased context length over the whole query window, not just the first five tracks.

6.3.2 Results

Table 6.3) Results of the Context Analysis

Split point	Median context	Median query	AUC	NDCG@5	Random NDCG@5	NDCG@5 lift
1	4	24	0.6148	0.7095	0.5857	0.1237
2	9	20	0.6272	0.7249	0.5965	0.1284
3	14	16	0.6327	0.7353	0.6106	0.1247
4	18	11	0.6337	0.7552	0.6399	0.1154
5	23	5	0.6171	0.838	0.7862	0.0518

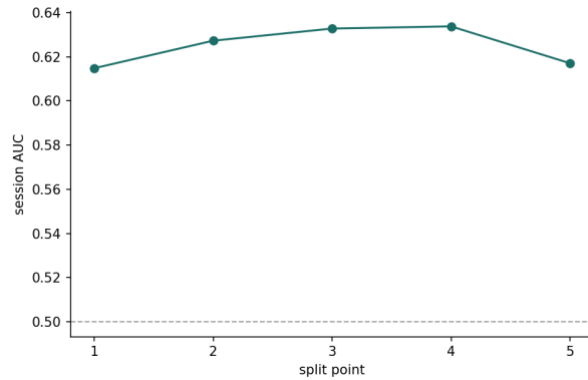


Figure 6.1) Graph depicting the results of the context analysis

As shown above, the AUC score increases up to the 4th split with median context length of 18 tracks. With an increase of 0.0189 compared to the first split. This is enough information to accept the hypothesis stated in Section 6.3.1. The increase in performance is concentrated early and saturates later on, with 66% of the improvement occurring from the first split to the second. This suggests that 66% of the useful information in a context arrives in the first 40% of tracks in a session, with subsequent tracks providing a comparatively small amount of information.

The decline in AUC at split 5 is due to a property of the metric rather than the performance of the model. At that point the query window contains a median of five tracks and roughly two skips (based on the random skip rate) so therefore has only six rankable pairs. This results in AUC becoming an unreliable metric for evaluating the performance, hence the lower score.

6.4 Limitations

6.4.1 Data

Due to the removal of the MSSD, Brost et al. (2019), in July 2024, the database used in comparable pieces of literature was not available for this research. Therefore, the dataset consists of 33 self-selecting volunteers on a single streaming platform, recruited through the researcher's own network. Despite an age-range of 18-68 years old the data is still subject to a limited population representativeness, with people from similar backgrounds consuming music in a similar fashion.

Audio features for each track in comparable works were drawn directly from the platform's interface. This was also discontinued, so these features were sourced from a third-party, which while substantial did not contain all data required to match each track in the volunteer's streaming data. This resulted in a loss of 10.1% of available data due to unmatched tracks. Recent releases likely make up a large portion of this drop-off due to the datasets most recent update occurring eight months ago. This surfaces an issue in deployment, as the model is trained on potentially stale data.

6.4.2 Session Filtering

The validity filters in Section 3.1.7 retain just 36.5% of constructed sessions upon execution. While the constraints of the filters are necessary as outlined in 3.1.7, they limit the available data and do not accurately depict what happens in real-life deployment. Currently this is working under the assumption that the listener will act in these specific ways when unsure of what they want to play.

6.4.3 Model Design

The constraint described in Section 3.2.2 excludes the most predictive signal from the feature inputs, reason start. This is corroborated under the premise of ranking in a live predictive scenario. This substantially reduces the scores produced by evaluation compared to what the data could achieve.

6.4.4 Comparison of Results

Although the results displayed in Chapter 6 provide positive comparisons to aid deduction of which Model/Loss combination performs most optimally. Many of the comparisons sit below the mean average of the standard deviation taken across the five seeds of each arm, 0.0062. Most notably the comparisons between the best performing loss function, PWTS, and the baseline BCE, with 0.0046 NDCG@5 on LSTM and 0.0019 NDCG@5 on SASRec. To truly conclude the results of this research a paired significance test would be needed to remove the between-session variance.

6.5 Reflection

Upon completion of this research there a few decisions and corrections I would have liked to have included given more time.

The first of which is the breadth of my volunteer search, rather than solely using my own network, reaching out to a broader population through the university, social media or other platforms would have increased the representativeness of the population used to train this model. This would also provide me with a larger database to explore the claim made by Dosovitskiy et al. (2021) that transformers underperform recurrent networks on moderate datasets.

Secondly, I would expand on the exploratory research I carried out into using the reason a track started as a feature only available during the training portion of the pipeline. Therefore using the feature with the highest contribution according to Meggetto et al. (2023) to explain interactions between positions in contextual data. I would also explore this option for the backbtn value of the reason end feature.

Thirdly, I would investigate the ways I could restrict the recurrent arm to the context segment so that I could directly compare the LSTM and SASRec models in the evaluation section.

Lastly, I would implement proper testing measures to the code as a large portion of my time was spent investigating errors I found as a consequence of finding unexpected numbers in the outputs of the pipeline.

7. Conclusion & Deployment

7.1 Conclusion

This research set out to rank the unplayed portion of a session of listening using predicted skip probability, while using only information knowable before a track is played to ensure the possibility of deployment. It also aimed to establish whether sequential models improve that ranking over simpler alternatives.

The first of these aims is answered positively. The strongest arm, an LSTM trained with PWTS, reaches 0.7846 NDCG@5 against a random-ordering floor of 0.6881, explaining 30.9% of the possible headroom above random ordering and increasing the average number of tracks listened to before a skip from 3.7 to 5.15 tracks.

The second of these aims is also answered positively with Logistic Regression scoring 0.7522 and Extra-Trees scoring 0.7585. This shows that the inclusion of the best sequential model increases the score by 0.0261 against the best performing baseline, which is a measurable improvement.

Subsequently, the conclusion that this research supports is that ranking a queue by predicted skip probability utilising sequential models measurably improves the order a listener receives and therefore their listening experience.

7.2 Deployment

As stated above, this research was carried out under the constraint of using information known only before a track is played so that it could be used in a practical setting. A demonstration of this is reported in Appendix D.

This demonstration reorders 3 randomly chosen sessions with randomly chosen context/query splits, held-out from training and prints the context and the remaining queue before and after the best model from evaluation (LSTM with PWTS) runs. As shown by each demo, the number of skips is measurably reduced in the top five songs, showing the success of the model.

Bibliography

- Adapa, S. (2019) 'Sequential modeling of sessions using recurrent neural networks for skip prediction'. arXiv preprint. WSDM Cup 2019 workshop.
- Alcrowd (2019) Spotify Sequential Skip Prediction Challenge — dataset access notice, 8 July 2024. Available at: <https://www.aicrowd.com/challenges/spotify-sequential-skip-prediction-challenge> (Accessed: 20 May 2026).
- Brost, B., Mehrotra, R. and Jehan, T. (2019) 'The Music Streaming Sessions Dataset', in The World Wide Web Conference (WWW 2019). New York: ACM, pp. 2594–2600.
- Catledge, L.D. and Pitkow, J.E. (1995) 'Characterizing browsing strategies in the World-Wide Web', Computer Networks and ISDN Systems, 27(6), pp. 1065–1073.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014) 'Learning phrase representations using RNN encoder–decoder for statistical machine translation', in Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP 2014). Stroudsburg: ACL, pp. 1724–1734.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J. and Houlsby, N. (2021) 'An image is worth 16x16 words: transformers for image recognition at scale', in 9th International Conference on Learning Representations (ICLR 2021).
- Ferrari Dacrema, M., Cremonesi, P. and Jannach, D. (2019) 'Are we really making much progress? A worrying analysis of recent neural recommendation approaches', in Proceedings of the 13th ACM Conference on Recommender Systems (RecSys 2019). New York: ACM, pp. 101–109.
- Hanley, J.A. and McNeil, B.J. (1982) 'The meaning and use of the area under a receiver operating characteristic (ROC) curve', Radiology, 143(1), pp. 29–36.
- Hidasi, B., Karatzoglou, A., Baltrunas, L. and Tikk, D. (2016) 'Session-based recommendations with recurrent neural networks', in 4th International Conference on Learning Representations (ICLR 2016).
- Hu, Y., Koren, Y. and Volinsky, C. (2008) 'Collaborative filtering for implicit feedback datasets', in 2008 Eighth IEEE International Conference on Data Mining (ICDM 2008). Piscataway: IEEE, pp. 263–272.
- Huang, Z. (2020) SASRec.pytorch. GitHub repository. Available at: <https://github.com/pmixer/SASRec.pytorch> (Accessed: 3 July 2026).
- Järvelin, K. and Kekäläinen, J. (2002) 'Cumulated gain-based evaluation of IR techniques', ACM Transactions on Information Systems, 20(4), pp. 422–446.
- Jeunen, O. and Goethals, B. (2019) 'Predicting sequential user behaviour with session-based recurrent neural networks', in WSDM Cup 2019 Workshop. Melbourne. Fifth place, Spotify Sequential Skip Prediction Challenge.
- Jin, S. and Lee, J. (2024) 'Effective music skip prediction based on late fusion architecture for user–interaction noise', Expert Systems with Applications, 238, 122098.
- Kang, W.-C. and McAuley, J. (2018) 'Self-attentive sequential recommendation', in 2018 IEEE International Conference on Data Mining (ICDM 2018). Piscataway: IEEE, pp. 197–206.

Meggetto, F., Revie, C., Levine, J. and Moshfeghi, Y. (2021) 'On skipping behaviour types in music streaming sessions', in Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM 2021). New York: ACM, pp. 3333–3337.

Meggetto, F., Revie, C., Levine, J. and Moshfeghi, Y. (2023) 'Why people skip music? On predicting music skips using deep reinforcement learning', in Proceedings of the 2023 Conference on Human Information Interaction and Retrieval (CHIIR 2023). New York: ACM, pp. 95–106.

Meng, Z., McCreadie, R., Macdonald, C. and Ounis, I. (2020) 'Exploring data splitting strategies for the evaluation of recommendation models', in Fourteenth ACM Conference on Recommender Systems (RecSys 2020). New York: ACM, pp. 681–686.

Moor, D., Yuan, Y., Mehrotra, R., Dai, Z. and Lalmas, M. (2023) 'Exploiting sequential music preferences via optimisation-based sequencing', in Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM 2023). New York: ACM, pp. 4759–4765.

Oramas, S., Nieto, O., Sordo, M. and Serra, X. (2017) 'A deep multimodal approach for cold-start music recommendation', in Proceedings of the 2nd Workshop on Deep Learning for Recommender Systems (DLRS 2017). New York: ACM, pp. 32–37.

ozefe (2025) Spotify Tracks & Audio Features Dataset [dataset]. Hugging Face. Available at: https://huggingface.co/datasets/ozefe/spotify_audio_features (Accessed: 28 May 2026).

Reimers, N. and Gurevych, I. (2017) 'Reporting score distributions makes a difference: performance study of LSTM-networks for sequence tagging', in Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017). Stroudsburg: ACL, pp. 338–348.

Schedl, M., Zamani, H., Chen, C.-W., Deldjoo, Y. and Elahi, M. (2018) 'Current challenges and visions in music recommender systems research', International Journal of Multimedia Information Retrieval, 7(2), pp. 95–116.

Seshadri, P., Shashaani, S. and Knees, P. (2024) 'Enhancing sequential music recommendation with negative feedback-informed contrastive learning', in Proceedings of the 18th ACM Conference on Recommender Systems (RecSys 2024). New York: ACM, pp. 1028–1032.

Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W. and Jiang, P. (2019) 'BERT4Rec: sequential recommendation with bidirectional encoder representations from transformer', in Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM 2019). New York: ACM, pp. 1441–1450.

van den Oord, A., Dieleman, S. and Schrauwen, B. (2013) 'Deep content-based music recommendation', in Advances in Neural Information Processing Systems 26 (NIPS 2013), pp. 2643–2651.

Wu, C., Wu, F., Huang, Y. and Xie, X. (2020) 'Neural news recommendation with negative feedback', CCF Transactions on Pervasive Computing and Interaction, 2(3), pp. 178–188.

Xu, C., Li, Q., Ge, J., Gao, J., Yang, X., Pei, C., Sun, F., Wu, J., Sun, H. and Ou, W. (2020) 'Privileged features distillation at Taobao recommendations', in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2020). New York: ACM, pp. 2590–2598.

Yi, X., Hong, L., Zhong, E., Liu, N.N. and Rajan, S. (2014) 'Beyond clicks: dwell time for personalization', in Proceedings of the 8th ACM Conference on Recommender Systems (RecSys 2014). New York: ACM, pp. 113–120.

Zhang, S., Chen, J., Li, C., Zhou, S., Shi, Q., Feng, Y., Chen, C. and Wang, C. (2025) 'Advancing loss functions in recommender systems: a comparative study with a Rényi divergence-based solution', Proceedings of the AAAI Conference on Artificial Intelligence, 39(12), pp. 13286–13294.

Zhu, H., Jin, J., Tan, C., Pan, F., Zeng, Y., Li, H. and Gai, K. (2017) 'Optimized cost per click in Taobao display advertising', in Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2017). New York: ACM, pp. 2191–2200.

Zinkevich, M. (2017) Rules of Machine Learning: Best Practices for ML Engineering. Google. Available at: <https://developers.google.com/machine-learning/guides/rules-of-ml> (Accessed: 3 July 2026).

Appendix A: Software Instructions

All code for this project is available at the following GitHub link.

<https://github.com/willmckenna15/Session-based-Music-queue-Reordering-System-Using-Skip-Behaviour-as-Implicit-Feedback>

Repository Layout

Directory	Contents
Data Scripts/	The five-stage data pipeline, orchestrated by <code>data_processing_main.py</code>
Model Scripts/	<code>Model_lib.py</code> (datasets, both architectures, evaluation metrics, training loop), <code>Loss_functions.py</code> , the four model entry points and their tuning scripts
Report Scripts/	<code>evaluate.py</code> , <code>context_analysis.py</code> , <code>demo_reorder.py</code> and results tables used in Chapter 6
Models/	Grid-search results, the selected configuration per arm, the trained checkpoints and <code>final_results.csv</code>
Info/	Volunteer consent form template & information sheets, Progress Tracker, confirmation of ethics approval, ethics form, Project Proposal
<code>run_lstm.sh</code> , <code>run_sasrec.sh</code> , <code>submit_all.sh</code>	SLURM job-array submission scripts

Order of execution of code

```
python3 Data Scripts/data_processing_main.py
./submit_all.sh bce
./submit_all.sh pwts
./submit_all.sh drrl
python3 Model Scripts/RNN.py --loss pwts
python3 Model Scripts/RNN.py --loss drrl
python3 Model Scripts/RNN.py --loss bce
python3 Model Scripts/SASRec.py --loss bce
python3 Model Scripts/SASRec.py --loss pwts
python3 Model Scripts/SASRec.py --loss drrl
python3 Model Scripts/Log_regression.py
python3 Model Scripts/ExtraTreesClassifier.py
python3 Report Scripts/evaluate.py
python3 Report Scripts/context_analysis.py
python3 Report Scripts/demo_reorder.py (3 times)
```

Environment. Python 3.14.3 with PyTorch 2.12.0, scikit-learn 1.8.0, pandas 3.0.2, NumPy 2.4.4 and PyArrow 24.0.0. Tuning and training were run as SLURM job arrays on the school's GPU cluster; initially the author attempted to run on Google colab, however this was costly, this is why there is still 'cuda' present in some of the scripts; every script can run on a single machine, hence the option of using 'mps' or 'cpu' but the 1,080 configuration search is too much workload for a laptop.

The volunteers' streaming histories and the processed datasets are excluded by .gitignore, in line with the ethics application (Appendix C): Raw Data/, Volunteer Data/ and all .parquet and .json files. All

scripts to carry out the listening analysis for the volunteers have also been gitignored as they are not relevant to this research.

Third party code: The self-attentive model in `Model_lib.py` is adapted from `SASRec.pytorch` Huang (2020), licensed under Apache 2.0; `PointWiseFeedForward` and the attention-block loop are unchanged from that source, and the modifications are described in Section 3.2.1. The `DrRLLoss` class in `Loss_funcitons.py` impletments the objective of Zhang et al. (2025) following their reference implementation, and shares no code with it.

Appendix B: AI Prompts & Tools

Claude:

5th August 2026 - Cluster Computing Setup (4 prompts)

The project required execution on a SLURM-manager HPC cluster, (after both the local machine and google colab experienced memory overload) a system with which the author had no prior experience. Claude was used to resolve infrastructure and configuration issues.

Prompts used:

- How do I access SLURM
- If I'm getting through one combination in 20 minutes using Google's L\$ for SASRec (162 combinations) how long will SLURM take
- When trying to connect using the SSH it says the connection is timing out
- I've downloaded the model files, these are the most up to date versions, how do I make this suitable for SLURM

Actions taken by the tool, at the author's direction: explained SLURM job submission commands and scripts structure; provided a compute-time estimate based on per-combination runtime; resolved an SSH connectivity timeout preventing cluster access from a Windows machine; adapted the author's existing training scripts to SLURM batch format (`#SBATCH` directives, module loading, job array configuration).

No model architecture, hyperparameter search space, evaluation metric, experimental design decision, or reported result was determined or affected by these iterations. All research decisions had been made prior to cluster setup; the tool was used only to implement the execution of pre-defined experiments on unfamiliar infrastructure.

14th August 2026 - version control (9 prompts)

Pushing the day's results from the laptop failed because the Machine's stored GitHub credentials belonged to a second account (willrosslyn) rather than the repository owner (willmckenna15).

Prompts:

- [pasted] `git push` → "Invalid username or token. Password authentication is not supported for Git operations."
- It's on willrosslyn
- [pasted] "Permission to willmckenna15/... denied to willrosslyn ... The requested URL returned error: 403"
- [pasted] `rpintf "protocol=https\nhost=github.com\nn" | git credential-osxkeychain erase` followed by `git push` → same 403. Option B is irrelevant it's a public repo
- [pasted] `git remote -v` output
- [pasted credential-helper configuration, `osxkeychain` set in two config files

- [pasted] git -c credential.helper=push → same 403
- Start back at the beginning, I just want to change the account on my laptop to willmckenna15 from willrosslyn
- [pasted] ! [rejected] main -> main (fetch first)

Actions take by the tool, at the author's direction: inspected git status, git remote -v and the credential-helper configuration; changed the remote from HTTPS to SSH; ran git fetch, compared the commits on the remote against the local branch, ran git pull --rebase, and ran git push.

No model, feature, loss function, evaluation decision or reported result was affected. The files pushed were the author's existing work.

Appendix C: Ethics

This research was submitted for ethical review to the School of Computing and Mathematical Sciences at Birkbeck University on the 13th of May 2026 under submission reference: **cmpbdwekq005x02ju3jbwishn**. It was classified as routine on the grounds that it collects personal music streaming data containing no sensitive categories, from volunteers over the age of 18 and proposes no risk to the participants or researcher. Approval was granted on the 13th of June 2026 by the School Research Ethics Officer. Data collection then began subsequently. The confirmation of approval can be found in the GitHub repo link in Appendix A.

Appendix D: Model Demonstration Results

Session aaab581f-9ddc-4019-bad2-aff350930484_171974				
35 tracks context 25 played 10 in the queue model: LSTM/PWTS (128 units)				
CONTEXT already played				
1. I Get Up	The Teskey Brothers	played		
2. I Took A Pill In Ibiza	Mike Posner	played		
3. I Took A Pill In Ibiza - Seeb Remi	Mike Posner	played		
4. Cooler Than Me - Single Mix	Mike Posner	played		
5. Tennessee Whiskey	Chris Stapleton	played		
6. Stick Season	Noah Kahan	played		
7. Stand by Me	Otis Redding	SKIPPED		
8. Death Row	Chris Stapleton	SKIPPED		
9. Never Tear Us Apart	The Teskey Brothers	SKIPPED		
10. Have You Ever Seen The Rain	Creedence Clearwater	played		
11. Sultans Of Swing	Dire Straits	played		
12. Sultans Of Swing	Dire Straits	played		
13. Could You Be Loved	Bob Marley & The Wai	played		
14. Scar Tissue	Red Hot Chili Pepper	played		
15. It Ain't Over 'Til It's Over	Lenny Kravitz	played		
16. Rocket Man (I Think It's Going To	Elton John	played		
17. Jolene	Dolly Parton	played		
18. Walk Of Life - Remastered 1996	Dire Straits	played		
19. (Sittin' On) the Dock of the Bay	Otis Redding	played		
20. Under the Bridge	Red Hot Chili Pepper	SKIPPED		
21. Go Your Own Way - 2004 Remaster	Fleetwood Mac	played		
22. Don't Go Breaking My Heart	Elton John	SKIPPED		
23. Romeo And Juliet	Dire Straits	played		
24. Son Of A Preacher Man	Dusty Springfield	played		
25. Roxanne	The Police	played		
QUEUE original order				
		P(skip)	actual	
1. Can't Stop	Red Hot Chili Pepper	0.132	played	
2. Stuck In The Middle With You	Stealers Wheel	0.118	played	
3. Tiny Dancer	Elton John	0.146	played	
4. Six Blade Knife	Dire Straits	0.219	SKIPPED	
5. Somebody To Love - Remastered 2011	Queen	0.180	played	
6. Don't Dream It's Over	Crowded House	0.191	played	
7. Californication	Red Hot Chili Pepper	0.275	played	
8. Dancing Queen	ABBA	0.196	played	
9. Bennie And The Jets - Remastered 2	Elton John	0.222	played	
10. Once Upon A Time In The West	Dire Straits	0.167	played	
QUEUE reordered by the model				
		P(skip)	actual	
1. Stuck In The Middle With You	Stealers Wheel	0.118	played <-	
2. Can't Stop	Red Hot Chili Pepper	0.132	played <-	
3. Tiny Dancer	Elton John	0.146	played <-	
4. Once Upon A Time In The West	Dire Straits	0.167	played <-	
5. Somebody To Love - Remastered 2011	Queen	0.180	played <-	
6. Don't Dream It's Over	Crowded House	0.191	played	
7. Dancing Queen	ABBA	0.196	played	
8. Six Blade Knife	Dire Straits	0.219	SKIPPED	
9. Bennie And The Jets - Remastered 2	Elton John	0.222	played	
10. Californication	Red Hot Chili Pepper	0.275	played	
NDCG@5 model 1.0000 original order 0.8539				
AUC model 0.7778				
Skips in the first 5: original 1/5 reordered 0/5				

Session fe645b5c-6424-49e4-b21a-99958d611b6c_295843							
27 tracks context 18 played 9 in the queue model: LSTM/PWTS (128 units)							
CONTEXT already played							
1. Amoeba	Clairo	played					
2. Summer's Over	Jordana	played					
3. You Make Loving Fun - 2004 Remaste	Fleetwood Mac	played					
4. Dirty Work	Steely Dan	played					
5. Amoeba	Clairo	played					
6. Open My Door	Alice Phoebe Lou	played					
7. Marcel	Her's	played					
8. What Once Was	Her's	played					
9. Seven	Men I Trust	played					
10. Numb	Men I Trust	played					
11. My Cherie Amour	Stevie Wonder	played					
12. Hold On - Remastered 2010	John Lennon	played					
13. Ghostride	Crumb	played					
14. 4EVER	Clairo	played					
15. Will I See You Again?	Thee Sacred Souls	played					
16. Ventura Highway	America	SKIPPED					
17. I Saw the Light	Todd Rundgren	played					
18. Add Up My Love	Clairo	played					
QUEUE original order				P(skip)	actual		
1. Better in the Dark	Jordana	0.124	SKIPPED				
2. Chateau (Feel Alright)	Djo	0.154	SKIPPED				
3. For Cryin' Out Loud!	FINNEAS	0.151	played				
4. She Needs Him	Her's	0.156	SKIPPED				
5. For Cryin' Out Loud!	FINNEAS	0.138	played				
6. Starfucker	FINNEAS	0.113	played				
7. Pool House	The Backseat Lovers	0.154	played				
8. Terrapin	Clairo	0.121	played				
9. Easier Said Than Done	Thee Sacred Souls	0.123	played				
QUEUE reordered by the model				P(skip)	actual		
1. Starfucker	FINNEAS	0.113	played <-				
2. Terrapin	Clairo	0.121	played <-				
3. Easier Said Than Done	Thee Sacred Souls	0.123	played <-				
4. Better in the Dark	Jordana	0.124	SKIPPED				
5. For Cryin' Out Loud!	FINNEAS	0.138	played <-				
6. For Cryin' Out Loud!	FINNEAS	0.151	played				
7. Chateau (Feel Alright)	Djo	0.154	SKIPPED				
8. Pool House	The Backseat Lovers	0.154	played				
9. She Needs Him	Her's	0.156	SKIPPED				
NDCG@5	model 0.8539		original order 0.3008				
AUC	model 0.7778						
Skips in the first 5: original 3/5				reordered 1/5			

Session ad265793-121a-482e-bbad-81b8c967acc2_182146							
24 tracks context 6 played 18 in the queue model: LSTM/PWTS (128 units)							
CONTEXT already played							
1. Cinderella's Big Score	Sonic Youth	SKIPPED					
2. Patria Amada, Idolatrada, Salve	Manduka	SKIPPED					
3. Games	Samir	SKIPPED					
4. Re-offender	Travis	SKIPPED					
5. Space to Bakersfield	Black Mountain	SKIPPED					
6. Love Buzz	Nirvana	played					
QUEUE original order							
		P(skip)	actual				
1. Disappearer	Sonic Youth	0.400	SKIPPED				
2. Tunic (Song For Karen)	Sonic Youth	0.576	SKIPPED				
3. My Friend Goo	Sonic Youth	0.486	SKIPPED				
4. The Cutter	Echo & the Bunnymen	0.747	SKIPPED				
5. My Life's All Right	Shira Small	0.802	SKIPPED				
6. Day Dreaming	Aretha Franklin	0.801	SKIPPED				
7. Kool Thing	Sonic Youth	0.739	played				
8. Shoot Your Gun	22-20s	0.471	played				
9. Evangeline - Remastered 2006	Cocteau Twins	0.722	played				
10. Lips Like Sugar	Echo & the Bunnymen	0.741	SKIPPED				
11. only shallow	my bloody valentine	0.659	SKIPPED				
12. North by North	The Bats	0.698	played				
13. Live with the Moon	Chayns	0.634	played				
14. Patria Amada, Idolatrada, Salve	Manduka	0.560	SKIPPED				
15. Cinderella's Big Score	Sonic Youth	0.690	SKIPPED				
16. Games	Samir	0.665	SKIPPED				
17. Re-offender	Travis	0.559	SKIPPED				
18. Space to Bakersfield	Black Mountain	0.601	SKIPPED				
QUEUE reordered by the model							
		P(skip)	actual				
1. Disappearer	Sonic Youth	0.400	SKIPPED				
2. Shoot Your Gun	22-20s	0.471	played <-				
3. My Friend Goo	Sonic Youth	0.486	SKIPPED				
4. Re-offender	Travis	0.559	SKIPPED				
5. Patria Amada, Idolatrada, Salve	Manduka	0.560	SKIPPED				
6. Tunic (Song For Karen)	Sonic Youth	0.576	SKIPPED				
7. Space to Bakersfield	Black Mountain	0.601	SKIPPED				
8. Live with the Moon	Chayns	0.634	played				
9. only shallow	my bloody valentine	0.659	SKIPPED				
10. Games	Samir	0.665	SKIPPED				
11. Cinderella's Big Score	Sonic Youth	0.690	SKIPPED				
12. North by North	The Bats	0.698	played				
13. Evangeline - Remastered 2006	Cocteau Twins	0.722	played				
14. Kool Thing	Sonic Youth	0.739	played				
15. Lips Like Sugar	Echo & the Bunnymen	0.741	SKIPPED				
16. The Cutter	Echo & the Bunnymen	0.747	SKIPPED				
17. Day Dreaming	Aretha Franklin	0.801	SKIPPED				
18. My Life's All Right	Shira Small	0.802	SKIPPED				
NDCG@5 model 0.2140 original order 0.0000							
AUC model 0.4769							
Skips in the first 5: original 5/5 reordered 4/5							